

Chủ Đề 9: Hệ Thống Khuyến Nghị Dựa Trên Hành Vi Duyệt Web Session-based Recommendation System

Phan Nguyễn Mai Phương – 25002711

Ngô Quốc Hoàng – 25001771

Bùi Thức Nam – 25003881

Trường Đại học Công nghiệp TP.HCM

Khoa Công nghệ Thông tin

GVHD: TS. Lê Thị Vĩnh Thanh

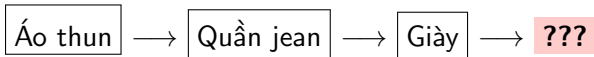
2024–2025

Nội dung trình bày

- 1 Giới thiệu bài toán
- 2 Dữ liệu và tiền xử lý
- 3 Thuật toán SKNN
- 4 Thuật toán GRU4Rec
- 5 Kết quả thí nghiệm
- 6 Kết luận

Tình huống:

- Khách hàng **ẩn danh** truy cập website bán hàng
- Không đăng nhập, không có tài khoản
- Trong 10 phút, click xem lần lượt:



Câu hỏi: Gợi ý sản phẩm nào tiếp theo?

Ứng dụng thực tế

- Amazon (khách vãng lai)
- Shopee, Tiki
- Trang tin tức
- Streaming video

Session-based Recommendation

Cho một phiên (session) $S = [i_1, i_2, \dots, i_t]$ là chuỗi sản phẩm người dùng đã tương tác theo thứ tự thời gian.

Mục tiêu: Dự đoán sản phẩm i_{t+1} mà người dùng sẽ tương tác tiếp theo.

- Input:** Chuỗi $[i_1, i_2, \dots, i_t]$ (các item đã click)
Output: Top-K item có khả năng được click tiếp theo
Đặc điểm: Không cần User ID, không cần lịch sử cũ

So sánh với Collaborative Filtering

Tiêu chí	CF truyền thống	Session-based RS
Cần ID người dùng?	Bắt buộc	Không cần
Dùng lịch sử dài hạn?	Có (toàn bộ)	Không
Cold-start user mới?	Không giải quyết	Xử lý được
Sở thích tức thời?	Kém	Tốt
Ứng dụng	Netflix, Spotify	Amazon, Shopee

Kết luận

Session-based RS phù hợp khi: người dùng ẩn danh, phiên ngắn, cần gợi ý real-time.

Tổng quan các phương pháp giải quyết bài toán

1. Phương pháp Thống kê

Dựa trên tần suất đồng xuất hiện và xác suất điều kiện.

- **POP**: Gợi ý theo mức độ phổ biến của các sản phẩm.
- **SKNN**: Tìm phiên lân cận theo độ tương đồng (Trọng tâm của nhóm).
- **Markov Chains**: Mô hình hóa sự chuyển dịch giữa các sản phẩm dưới dạng xsđk.

2. Mạng Hồi quy

Mô hình hóa chuỗi tương tác tuần tự có phụ thuộc thời gian.

- **GRU4Rec**: Dùng mạng GRU học vector trạng thái ẩn của phiên (Trọng của tâm nhóm).
- **NARM**: Tích hợp *Attention* phân tách mục đích toàn cục và sở thích tức thời.

3. Mô hình nâng cao

Mô hình hóa cấu trúc hình học đồ thị hoặc Self-Attention hai chiều.

- **SR-GNN**: Chuyển phiên thành đồ thị, dùng GNN học quan hệ đan xen.
- **BERT4Rec**: Sử dụng Transformer để song song hóa, nắm bắt ngữ cảnh.

Tiến trình tiến hóa: Heuristic/Statistical → Recurrent Neural Networks → Graph/Self-Attention Mechanisms.

Cấu trúc dữ liệu Session

SessionID	Timestamp	ItemID	Action
S001	09:01:03	I042	click
S001	09:03:45	I087	click
S001	09:07:12	I015	click
S001	09:09:55	I033	click
S002	10:15:00	I201	click
S002	10:18:30	I202	click

- Phiên S001 → chuỗi: [I042, I087, I015, I033]
- Phiên S002 → chuỗi: [I201, I202]
- Mỗi phiên là một “câu chuyện” **độc lập**

Dataset: Yoochoose (RecSys Challenge 2015)

Thuộc tính	Giá trị
Tổng click	33 triệu
Số phiên	9.2 triệu
Số sản phẩm	52,739
Lĩnh vực	E-commerce
<i>Sau lấy mẫu 1/64:</i>	
Số phiên (mẫu)	144,527
Sau lọc	116,267
Train	108,263
Test	8,004

Tiền xử lý

- 1 Chia train/test theo **thời gian** (90/10)
- 2 Lọc item hiếm (< 5 lần, **chỉ trên train**)
- 3 Lọc độ dài phiên $\in [2, 20]$

Lưu ý

Chia theo thời gian **TRƯỚC**, rồi lọc item hiếm chỉ trên train $\rightarrow$ tránh data leakage.

Session K-Nearest Neighbors (SKNN)

Tìm K phiên lịch sử **tương tự nhất** với phiên hiện tại, sau đó tổng hợp các item từ những phiên đó để sinh khuyến nghị.

Ký hiệu:

$S_q = [i_1, i_2, \dots, i_t]$	phiên truy vấn (phiên hiện tại)
$\mathcal{S} = \{S_1, S_2, \dots, S_N\}$	tập tất cả phiên lịch sử
K	số hàng xóm gần nhất (hyperparameter)
$KNN(S_q)$	K phiên có similarity cao nhất với S_q

SKNN – Công thức

1. Đo độ tương tự giữa hai phiên:

$$\text{sim}(S_q, S_n) = \frac{|S_q \cap S_n|}{\sqrt{|S_q| \times |S_n|}} \quad (1)$$

$|S_q \cap S_n|$: số item trùng nhau. Giá trị $\in [0, 1]$.

2. Tính điểm cho item ứng viên i ($i \notin S_q$):

$$\text{score}(i) = \sum_{S_n \in \text{KNN}(S_q)} \text{sim}(S_q, S_n) \times \mathbb{K}[i \in S_n] \quad (2)$$

$\mathbb{K}[i \in S_n] = 1$ nếu item i có trong phiên S_n , ngược lại $= 0$.

3. Sinh khuyến nghị:

$$\text{Top-N} = \underset{i \notin S_q}{\text{argmax-N}} \text{ score}(i) \quad (3)$$

SKNN – Mã giả thuật toán

Algorithm 1: SKNN Prediction

Input: S_q (current session), S (history sessions), K , N

Output: Top- N recommended items

```
1: for each  $S_n$  in  $S$  do
2:    $\text{sim}(S_q, S_n) = |S_q \cap S_n| / \sqrt{|S_q| \cdot |S_n|}$ 
3: end for
4: sort  $S$  by  $\text{sim}$  (descending)
5:  $\text{KNN} \leftarrow$  top- $K$  sessions with  $\text{sim} > 0$ 
6: for each item  $i$  in  $\text{KNN}$  with  $i \notin S_q$  do
7:    $\text{score}(i) = \sum (\text{sim}(S_q, S_n) * 1[i \in S_n])$ 
8: end for
9: sort items by  $\text{score}$  (descending)
10: return Top- $N$  items
```

Complexity: $O(|S| \times |S_q|)$ – dùng inverted index để tăng tốc.

Tối ưu hóa SKNN: Kỹ thuật Inverted Index

1. Vấn đề thất cổ chai tính toán

Độ phức tạp gốc của SKNN là $O(|S| \times |S_q|)$. Nếu hệ thống có hàng triệu phiên lịch sử S , việc duyệt tuần tự để so sánh với phiên hiện tại S_q sẽ làm hệ thống tê liệt, không thể đáp ứng thời gian thực.

2. Giải pháp: Chỉ mục đảo (Inverted Index)

Thay vì lưu trữ xuôi chiều (*Phiên* $\rightarrow$ *Sản phẩm*), ta xây dựng một từ điển ngược: **Sản phẩm** $\rightarrow$ **Danh sách các phiên chứa nó**. Khi có phiên truy vấn, thuật toán chỉ cần gọi ra các phiên lịch sử có chứa ít nhất 1 sản phẩm trùng khớp.

3. Minh họa cấu trúc từ điển ngược

Sản phẩm (Item)	Danh sách các phiên lịch sử chứa Item
Item A	$\{S_1, S_5, S_{102}, S_{890}\}$
Item B	$\{S_2, S_5\}$
Item C	$\{S_1, S_3, S_4, S_{102}\}$

Hiệu quả: Giảm không gian tìm kiếm từ hàng triệu phiên xuống chỉ còn vài trăm phiên ứng viên, giúp SKNN đạt tốc độ phản hồi chỉ vài mili-giây.

SKNN – Ví dụ tính tay

Cho: $S_q = \{I1, I2, I3\}$, $K = 3$

Bước 1 – Tính similarity:

Phiên	Giao với S_q	Công thức	sim
$S_1 = \{I1, I2, I3, I4\}$	$\{I1, I2, I3\}$	$3/\sqrt{3 \times 4}$	0.866
$S_2 = \{I1, I2, I5\}$	$\{I1, I2\}$	$2/\sqrt{3 \times 3}$	0.667
$S_3 = \{I2, I3, I6\}$	$\{I2, I3\}$	$2/\sqrt{3 \times 3}$	0.667
$S_4 = \{I5, I6, I7\}$	$\emptyset$	–	0.000

Bước 2 – KNN: $\{S_1, S_2, S_3\}$

Bước 3 – Score (loại $\{I1, I2, I3\}$):

- $\text{score}(I4) = 0.866 \times 1 + 0.667 \times 0 + 0.667 \times 0 = \mathbf{0.866}$
- $\text{score}(I5) = 0 + 0.667 \times 1 + 0 = 0.667$
- $\text{score}(I6) = 0 + 0 + 0.667 \times 1 = 0.667$

Bước 4 – Khuyến nghị: $[I4, I5, I6]$ ✓

Ý tưởng chính (Hidasi et al., ICLR 2016)

Dùng mạng hồi quy GRU để học biểu diễn ẩn của chuỗi hành vi, từ đó dự đoán item tiếp theo.

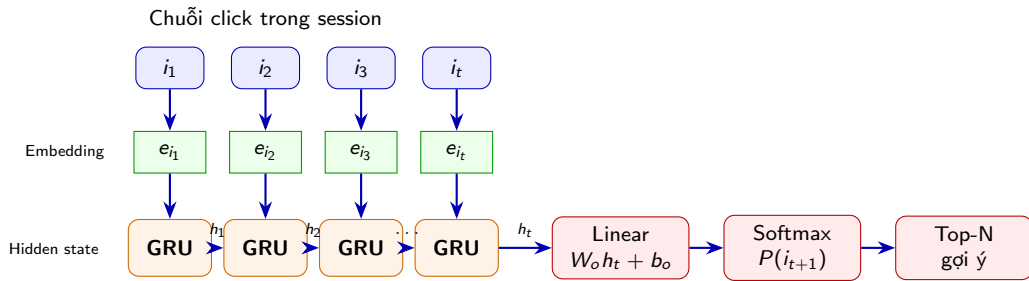
Ký hiệu:

M	tổng số item trong catalog
$E \in \mathbb{R}^{M \times d}$	ma trận embedding ($d = 64$)
$e_t = E[i_t]$	vector embedding của item i_t
$h_t \in \mathbb{R}^H$	hidden state tại bước t ($H = 128$)
$W_o \in \mathbb{R}^{H \times M}$	ma trận output

Pipeline:

$$i_t \xrightarrow{\text{Embed}} e_t \xrightarrow{\text{GRU}} h_t \xrightarrow{\text{Linear}} \hat{y}_t \in \mathbb{R}^M \xrightarrow{\text{softmax}} P(i_{t+1})$$

GRU4Rec – Minh họa kiến trúc



- GRU cập nhật trạng thái ẩn h_t theo thứ tự click trong phiên.
- Hidden state cuối được đưa qua Linear + Softmax để xếp hạng item tiếp theo.

GRU – Công thức chi tiết

Tại mỗi bước t , nhận input $x_t = e_{i_t}$ và hidden state h_{t-1} :

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad (\text{Update gate}) \quad (4)$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad (\text{Reset gate}) \quad (5)$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h) \quad (\text{Candidate}) \quad (6)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (\text{New state}) \quad (7)$$

Ký hiệu	Ý nghĩa
$z_t \approx 0$	Giữ nguyên h_{t-1} (nhớ quá khứ)
$z_t \approx 1$	Thay bằng $\tilde{h}_t$ (học từ hiện tại)
r_t	Kiểm soát quá khứ ảnh hưởng đến candidate
$\odot$	Phép nhân element-wise

Sau khi GRU xử lý chuỗi, lấy hidden state cuối h_t :

Logits:

$$\hat{y}_t = W_o \cdot h_t + b_o \in \mathbb{R}^M \quad (8)$$

Xác suất (softmax):

$$P(i_{t+1} = j \mid S_{1:t}) = \frac{\exp(\hat{y}_t[j])}{\sum_{k=1}^M \exp(\hat{y}_t[k])} \quad (9)$$

Khuyến nghị: Lấy Top-N item có $\hat{y}_t[j]$ cao nhất (loại item đã xem).

Cross-Entropy Loss:

$$\mathcal{L}_{CE} = -\frac{1}{|\mathcal{D}|} \sum_{(S, i^+) \in \mathcal{D}} \log P(i^+ | S) \quad (10)$$

i^+ : item thực sự được click tiếp theo (ground truth).

BPR Loss (Bayesian Personalized Ranking):

$$\mathcal{L}_{BPR} = -\frac{1}{|\mathcal{D}|} \sum_{(S, i^+)} \frac{1}{N_s} \sum_{j=1}^{N_s} \log \sigma(\hat{y}[i^+] - \hat{y}[j]) \quad (11)$$

j : negative sample (item ngẫu nhiên), σ : sigmoid.

Mục tiêu

Đảm bảo score item đúng (i^+) luôn cao hơn item sai (j).

GRU4Rec – Mã giả thuật toán

Algorithm 2: GRU4Rec Training

Input: sessions S , learning rate η , epochs E

Output: model parameters θ

```
1: initialize  $\theta \sim N(0, 0.01)$ 
2: for epoch = 1 to  $E$  do
3:   for each mini-batch  $B$  in  $S$  do
4:     for each session  $[i_1, \dots, i_T]$  in  $B$  do
5:        $h_0 \leftarrow 0$ 
6:       for  $t = 1$  to  $T-1$  do
7:          $e_t \leftarrow E[i_t]$  // embedding
8:          $h_t \leftarrow \text{GRU}(e_t, h_{t-1})$  // forward
9:          $y_t \leftarrow W_0 h_t + b_0$  // logits
10:         $L_t \leftarrow -\log P(i_{t+1} | y_t)$ 
11:      end for
12:    end for
13:     $\theta \leftarrow \theta - \eta * \text{grad}(L)$ 
14:  end for
15: end for
```

Algorithm 3: GRU4Rec Inference

Input: $S_q = [i_1, \dots, i_t]$, trained model, Top-N

Output: N item khuyến nghị

```
1:  $h_0 \leftarrow 0$ 
2: for  $k = 1$  to  $t$  do
3:    $e_k \leftarrow E[i_k]$  // embedding lookup
4:    $h_k \leftarrow \text{GRU}(e_k, h_{(k-1)})$  // forward GRU
5: end for
6:  $y \leftarrow W_o * h_t + b_o$  // score mọi item
7:  $y[i] \leftarrow -\text{inf}$ , for all  $i$  in  $S_q$  // loại đã xem
8: return Top-N item có  $y[j]$  cao nhất
```

Complexity:

- Training: $O(E \times |S| \times \bar{T} \times (d \cdot H + H \cdot M))$
- Inference: $O(t \times d \cdot H + H \times M)$ – **real-time**

So sánh ưu và nhược điểm: SKNN vs GRU4Rec

SKNN (Memory-based)

Ưu điểm:

- **Đơn giản & Minh bạch:** Dễ giải thích kết quả gợi ý thông qua các hàng xóm tương tự.
- **Chi phí thấp:** Triển khai nhanh, không cần huấn luyện phức tạp và không đòi hỏi phần cứng mạnh (GPU).

Nhược điểm:

- **Độ trễ khi Inference:** Tốn chi phí tính toán khi dự đoán vì phải quét lại tập dữ liệu lịch sử để tìm độ tương tự.

GRU4Rec (Deep Learning)

Ưu điểm:

- **Học chuỗi tình tế:** Học được biểu diễn ẩn và bắt được thứ tự hành vi click theo thời gian một cách chính xác.
- **Inference nhanh:** Tốc độ dự đoán cực kỳ nhanh khi triển khai thực tế (chỉ chạy qua mô hình đã tối ưu).

Nhược điểm:

- **Khối lượng tính toán:** Cần nhiều thời gian/epoch và lượng lớn dữ liệu để huấn luyện.
- **Tốn tài nguyên:** Đòi hỏi thiết lập hạ tầng phần cứng mạnh (GPU).

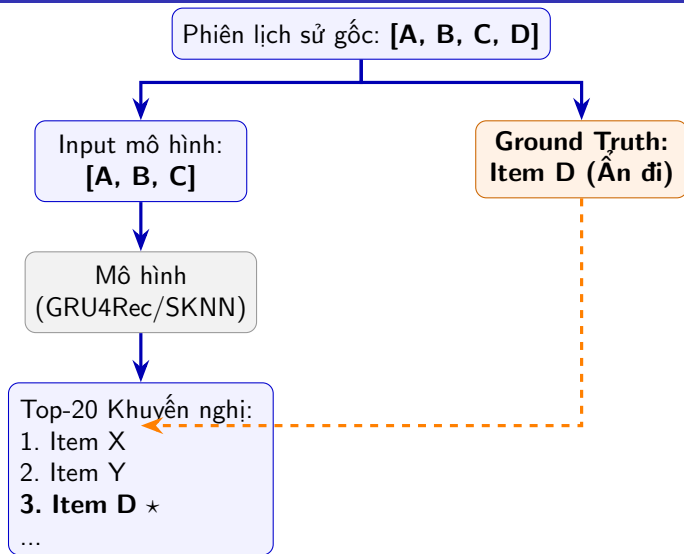
Metrics đánh giá

Chỉ số	Công thức	Ý nghĩa
Recall@20	$\frac{ \text{hit} }{ \text{test} }$	Tỉ lệ phiên có item đúng trong Top-20
MRR@20	$\frac{1}{N} \sum \frac{1}{\text{rank}_i}$	Trung bình nghịch đảo vị trí item đúng

Chiến lược đánh giá: Leave-one-out

- Phiên test: $[A, B, C, D]$
- Input cho mô hình: $[A, B, C]$
- Đáp án: D
- Kiểm tra: D có nằm trong Top-20 gợi ý không?

Minh họa chiến lược đánh giá Leave-one-out (LOO)



Cơ chế tính toán Metrics trên ví dụ thực tế

- **Recall@20 (Độ phủ):** Do đáp án chính xác (Item **D**) có xuất hiện trong danh sách Top-20 khuyến nghị đầu ra $\rightarrow$ Phiên này được ghi nhận là **1 Hit** (Thành công).
- **MRR@20 (Thứ hạng nghịch đảo):** Không chỉ cần đoán trúng, mô hình cần đưa đáp án lên vị trí càng cao càng tốt.

$$\text{Vị trí xuất hiện của } D = 3 \implies \text{Điểm số thu về (RR)} = \frac{1}{3} \approx 0.333$$

$\Rightarrow$ Điểm số chung cuộc của mô hình là trung bình cộng Recall và MRR của tất cả các phiên kiểm thử.

Kết quả trên Yoochoose (1/64)

Mô hình	Recall@20	MRR@20
Popularity Baseline	0.0187 (1.87%)	0.0041
SKNN (k=500)	0.2952 (29.5%)	0.1349
GRU4Rec (10 epoch)	0.2796 (28.0%)	0.1262

Nhận xét

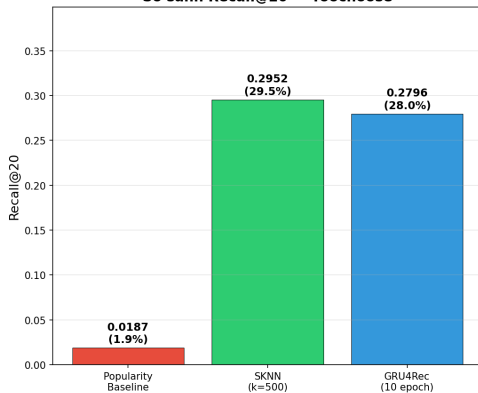
- SKNN vượt Popularity **+27.65 điểm**
- SKNN nhỉnh hơn GRU4Rec **+1.56 điểm**
- Cả hai đều vượt xa baseline

Ý nghĩa

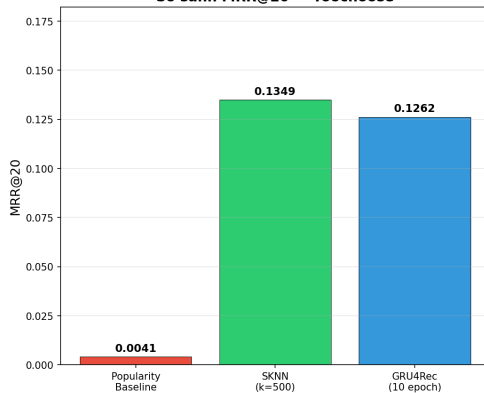
Tận dụng thông tin chuỗi trong session thực sự có giá trị lớn so với chỉ dùng popularity.

Biểu đồ so sánh

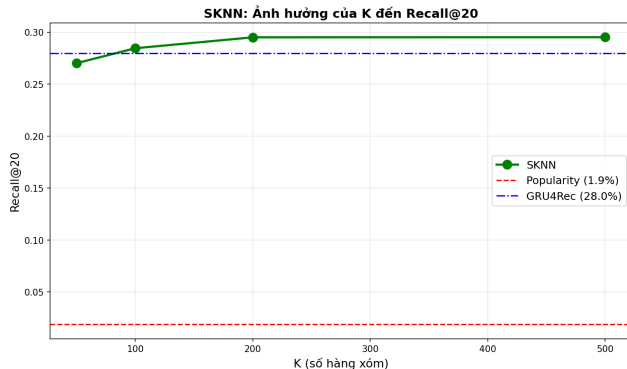
So sánh Recall@20 — Yoochoose



So sánh MRR@20 — Yoochoose



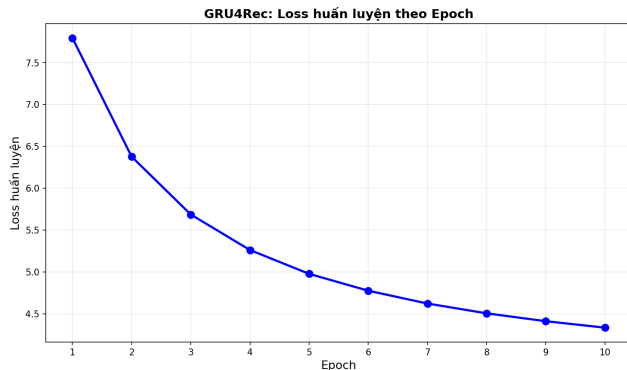
Ảnh hưởng của K (SKNN)



- K nhỏ (50): ít hàng xóm → bỏ sót item tốt
- K lớn: Recall tăng dần rồi **bão hòa** (200→500 gần như không đổi)
- **K=500**: tốt nhất trong dải đã thử

K	Recall@20
50	0.2702
100	0.2846
200	0.2951
500	0.2952

GRU4Rec – Training Loss



- Loss giảm đều qua 10 epoch
- Chưa hội tụ hoàn toàn
- Có thể cải thiện thêm với:
 - Nhiều epoch hơn (20–30)
 - Learning rate scheduling
 - Larger hidden size

Cấu hình

Embedding: 64, Hidden: 128

Optimizer: Adam, lr=0.001

Batch size: 256

So sánh với các thí nghiệm tương tự

Tiêu chí so sánh	Thí nghiệm của Nhóm	Tham chiếu 1	Tham chiếu 2
Tập dữ liệu	Yoochoose(Mẫu 1/64)	Diginetica	Yoochoose(Gốc 1/1)
Nguồn / Tác giả	TN thực tế (nhóm)	VD minh họa (cô)	Hidasi et al. (2016)
Số phiên Train	108,263	164,401	7,966,257
Số sản phẩm (Items)	10,435	43,097	37,483
Recall@20			
– Popularity Baseline	1.87%	28.41%	0.50%
– SKNN (k=500)	29.52%	51.23%	50.65% (Item-KNN)
– GRU4Rec	27.96% (10 epochs)	58.34% (5 epochs)	- - -
MRR@20			
– Popularity Baseline	0.0041	0.1023	0.0012
– SKNN (k=500)	0.1349	0.2187	0.2048 (Item-KNN)
– GRU4Rec	0.1262	0.2563	- - -

Lưu ý: Các tham chiếu chỉ mang tính chất tham khảo do chưa so sánh đúng mô hình, tập dữ liệu

Nội dung	Điểm chính
Bài toán Khác CF	Dự đoán item tiếp theo từ chuỗi hành vi phiên hiện tại Không cần User ID, phù hợp phiên ẩn danh/cold-start user
SKNN	Đơn giản, hiệu quả, không cần GPU
GRU4Rec	Học được thứ tự click; cần thêm dữ liệu/epoch để cải thiện
Kết quả	Popularity 1.87% → GRU4Rec 28.0% → SKNN 29.5%

Bài học thực nghiệm

Nên bắt đầu với Popularity Baseline và SKNN. Nếu SKNN chưa vượt rõ Popularity, cần kiểm tra lại dữ liệu và protocol trước khi đầu tư vào mô hình học sâu.

Hướng	Mô tả	Độ khó
NARM	Thêm Attention vào GRU	★ ★ ★
SR-GNN	Session dạng đồ thị + GNN	★ ★ ★★
BERT4Rec	BERT bidirectional cho session	★ ★ ★★
Tiếng Việt	Dataset từ Tiki/Shopee	★ ★ ★

Lộ trình phát triển các mô hình Session-based nâng cao



- **Hạn chế của GRU4Rec (Baseline):** Chỉ học chuỗi hành vi một chiều ($\rightarrow$) và coi trọng số (tầm quan trọng) các lượt click trong một phiên là ngang nhau.
- **NARM (Khắc phục tính đồng đều):** Tích hợp cơ chế **Attention** vào mạng RNN (GRU). Giúp mô hình tự động nhận diện và "tập trung" trọng số vào các click cốt lõi thể hiện ý định chính của phiên.
- **BERT4Rec (Khắc phục tính một chiều):** Ứng dụng kiến trúc **Transformer (Bidirectional)**. Cho phép học ngữ cảnh hai chiều đồng thời, thay vì chỉ phụ thuộc vào các hành vi đứng trước.
- **SR-GNN (Khắc phục cấu trúc tuần tự đơn điệu):** Chuyển đổi chuỗi phiên thành cấu trúc **Đồ thị (Graph)**. Khai thác sâu các mối quan hệ chuyển tiếp phức tạp và phi tuyến tính giữa các item trong và giữa các phiên.

$\Rightarrow$ Việc chuyển dịch từ học tuần tự sang học ngữ cảnh hai chiều và cấu trúc đồ thị là xu hướng tất yếu giúp tối ưu hóa độ chính xác gợi ý.

- ❶ Hidasi B. et al. (2016). *Session-based Recommendations with Recurrent Neural Networks*. ICLR.
<https://arxiv.org/abs/1511.06939>
- ❷ Ludewig M., Jannach D. (2018). *Evaluation of Session-based Recommendation Algorithms*. UMUAI.
<https://arxiv.org/abs/1803.09587>
- ❸ Jannach D. et al. (2017). *When RNNs meet the Neighborhood for Session-Based Recommendation*. RecSys.
- ❹ Li J. et al. (2017). *Neural Attentive Session-based Recommendation*. CIKM.
- ❺ Dataset: Yoochoose – RecSys Challenge 2015.
<https://www.kaggle.com/datasets/chadgostopp/recsys-challenge-2015>

Cảm ơn thầy/cô đã lắng nghe!

Q & A